

Retrieval-Augmented Generation, commonly called RAG, combines vector search with large language models to generate accurate responses.

Traditional language models rely mostly on training data and may hallucinate or provide outdated information.

RAG systems improve reliability by retrieving relevant information from external documents before generating an answer.

The retrieval process usually starts by converting documents into embeddings.

Embeddings are numerical vector representations of text that capture semantic meaning.

Models such as OpenAI text-embedding-3-small generate embeddings for documents and user queries.

These vectors are stored inside vector databases such as FAISS, Pinecone, or Weaviate.

When a user asks a question, the system converts the query into an embedding and performs similarity search.

The most relevant document chunks are retrieved and added to the prompt sent to the language model.

This allows the language model to answer using context from company documents, PDFs, or knowledge bases.

Chunking is another important concept in RAG systems.

Large documents are split into smaller overlapping chunks to improve retrieval accuracy.

Smaller chunks help the vector search identify highly relevant information instead of returning entire large documents.

RAG systems are widely used in enterprise AI applications.

Examples include customer support chatbots, document assistants, coding copilots, and internal knowledge search systems.

By combining retrieval with generation, RAG architectures provide grounded and context-aware responses while reducing hallucinations.